

Refresh Tokens e OAuth 2.0

DIM0547 — Sprint 3

Prof. Fernando · UFRN · 2026.1

De onde paramos

Aula passada: a API tem **login** e um **middleware** que valida o JWT.

Mas deixamos uma tensão em aberto:

Token de acesso curto (15 min) → seguro, mas...
→ usuário deslogado a cada 15 min?

Hoje resolvemos	Hoje exploramos
Como manter a sessão viva sem reduzir a segurança	Como delegar autenticação a um provedor externo
Refresh tokens	OAuth 2.0

Plano da aula

1. O dilema: token curto vs experiência do usuário ~5 min
2. Refresh token: dois tokens, dois papéis ~15 min
3. O fluxo de renovação na prática ~15 min
4. Rotação e revogação de refresh tokens ~10 min
5. OAuth 2.0: vocabulário e os atores ~10 min
6. Authorization Code e Client Credentials ~15 min

0 dilema do tempo de vida

exp curto (minutos)	exp longo (dias)
└ vazamento dura pouco	└ vazamento dura muito
└ usuário relogga toda hora (péssima UX)	└ usuário fica logado (boa UX, péssima segurança)

Os dois objetivos — **segurança** e **boa experiência** — puxam para lados opostos. Um único token não resolve.

A saída é **separar responsabilidades**: um token para *acessar* (curto, usado o tempo todo) e outro para *renovar* (longo, usado raramente). Cada um otimiza um objetivo.

Dois tokens, dois papéis

	Access token	Refresh token
Função	Provar identidade em cada request	Obter um novo access token
Tempo de vida	Curto — 5 a 15 min	Longo — dias a semanas
Usado	Em toda requisição	Só quando o access expira
Onde fica	Memória do cliente	Armazenamento mais protegido
É stateless?	Sim — JWT autocontido	Não — guardado no banco

A assimetria é o ponto: o token que viaja muito (access) é o que dura pouco. O token que dura muito (refresh) quase não viaja — e, por ser guardado no banco, **pode ser revogado**.

0 fluxo com refresh



Para o usuário, nada acontece — o cliente renova **nos bastidores**. Ele só digita a senha de novo quando o **refresh** também expira (7 dias depois).

Por que o refresh fica no banco

O access token é JWT puro: stateless, validado pela assinatura. O refresh é diferente — **precisa** de estado:

```
CREATE TABLE refresh_tokens (  
  id          SERIAL PRIMARY KEY,  
  user_id     INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  token_hash  TEXT NOT NULL,           -- hash, nunca o token cru  
  expires_at  TIMESTAMPTZ NOT NULL,  
  revoked     BOOLEAN NOT NULL DEFAULT false  
);
```

Repare na modelagem 1:N da Sprint 2 — um usuário tem muitos refresh tokens (um por dispositivo). E guardamos o **hash** do token, não o token: mesma lógica do bcrypt para senhas. Vazou o banco? O atacante não tem os tokens.

0 handler de refresh

```
func (h *AuthHandler) Refresh(w http.ResponseWriter, r *http.Request) {
    var req RefreshRequest
    json.NewDecoder(r.Body).Decode(&req)

    rt, err := h.repo.FindRefreshToken(r.Context(), hash(req.RefreshToken))
    if err != nil || rt.Revoked || time.Now().After(rt.ExpiresAt) {
        http.Error(w, "refresh inválido", http.StatusUnauthorized)
        return
    }
    user, _ := h.repo.FindByID(r.Context(), rt.UserID)
    newAccess, _ := generateAccessToken(user)
    writeJSON(w, map[string]string{"access_token": newAccess})
}
```

Três checagens antes de emitir um token novo: o refresh **existe** no banco, **não foi revogado** e **não expirou**. Falhou alguma → 401.

Rotação de refresh tokens

Boa prática: a cada `/refresh`, **invalidar o refresh usado** e emitir um novo.

```
/refresh com RT-1 → revoga RT-1, emite RT-2 (+ access novo)
/refresh com RT-2 → revoga RT-2, emite RT-3
/refresh com RT-1 → RT-1 já revogado → 401 + ALERTA
```

Se um RT já revogado reaparece, é sinal de **token roubado**: ou o cliente legítimo ou o atacante está usando uma cópia antiga. A resposta defensiva é revogar **toda a família** de tokens daquele usuário — forçar novo login.

Revogação: o logout que funciona

Lembra do problema da aula passada — JWT não dá para revogar? O refresh token **resolve isso**:

```
// POST /logout
func (h *AuthHandler) Logout(w http.ResponseWriter, r *http.Request) {
    var req RefreshRequest
    json.NewDecoder(r.Body).Decode(&req)
    h.repo.RevokeRefreshToken(r.Context(), hash(req.RefreshToken))
    w.WriteHeader(http.StatusNoContent)
}
```

Token	Após o logout
Access token	Ainda válido — mas expira em minutos
Refresh token	Revogado imediatamente — não renova mais

Logout não é instantâneo no access token, mas a janela é curta (minutos). É o trade-off consciente do modelo stateless. Para revogação **imediate e total**, só com sessão no servidor.

OAuth 2.0

"Entre com o Google" — você usa o app, mas **não dá sua senha do Google para o app**.

Sem OAuth: app pede sua senha do Google → app TEM sua senha 🙏
Com OAuth: Google autentica você → devolve um TOKEN ao app
(app nunca vê sua senha)

OAuth 2.0 é um **protocolo de autorização delegada**. Ele permite que um app obtenha acesso limitado a um recurso **sem** receber as credenciais do dono. A senha fica só com o provedor.

OAuth 2.0: os quatro atores

Ator	Papel	Exemplo
Resource Owner	O dono dos dados	Você
Client	A aplicação que quer acesso	O app que você está usando
Authorization Server	Autentica e emite tokens	O Google
Resource Server	Guarda os dados protegidos	API do Google (contatos, agenda)

Note que **autenticação** e **dados** podem estar em servidores diferentes. O Authorization Server só cuida de identidade e tokens; o Resource Server confia nos tokens que ele emite.

Authorization Code Flow

O fluxo padrão para apps com usuário na frente da tela:

1. App redireciona o usuário ao Authorization Server
↓
2. Usuário faz login NO PROVEDOR e autoriza o acesso
↓
3. Provedor redireciona de volta ao app com um CODE curto
↓
4. App troca o CODE (+ client_secret) por um access_token
| ← essa troca é servidor-a-servidor, fora do navegador
↓
5. App usa o access_token para chamar a API do provedor

Por que um `code` intermediário e não o token direto? O `code` viaja pelo **navegador** (menos seguro); a troca por token é **servidor-a-servidor**. O token nunca fica exposto na URL.

Client Credentials Flow

Quando **não há usuário** — um serviço falando com outro:



Authorization Code	Client Credentials
Tem usuário humano	Máquina-a-máquina
Pede consentimento na tela	Sem tela, sem consentimento
code → token	credenciais → token direto
App acessa dados do usuário	Serviço acessa recursos próprios

Exemplo no projeto: um cron job que chama sua API à noite. Não há "usuário" — é o Client Credentials flow. As credenciais identificam o **serviço**, não uma pessoa.

JWT e OAuth: como se encaixam

São coisas **diferentes** que costumam aparecer juntas — não confunda:

	É um...	Responde
JWT	Formato de token	"Como representar a identidade?"
OAuth 2.0	Protocolo de autorização	"Como obter um token sem dar a senha?"

O `access_token` que o OAuth entrega **muitas vezes é um JWT** — mas não precisa ser. E o JWT da aula passada funciona **sem** OAuth nenhum. Um é o "o quê", o outro é o "como obter". Misturar os dois conceitos é uma confusão comum.

Escopos: autorização granular

OAuth carrega **escopos** — o que exatamente o token permite:

```
scope: "contacts:read"           → só leitura de contatos  
scope: "contacts:read contacts:write" → leitura e escrita
```

```
func RequireScope(scope string) func(http.Handler) http.Handler {  
    return func(next http.Handler) http.Handler {  
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {  
            claims := r.Context().Value(userKey).(jwt.MapClaims)  
            if !hasScope(claims["scope"], scope) {  
                http.Error(w, "escopo insuficiente", http.StatusForbidden)  
                return  
            }  
            next.ServeHTTP(w, r)  
        })  
    }  
}
```

401 Unauthorized = "não sei quem você é". 403 Forbidden = "sei quem você é, mas você não pode isso". Escopo insuficiente é **403**, não 401.

Erros comuns nesta aula

Sintoma	Causa provável
Refresh "funciona" mas access nunca renova	Cliente não troca o token antigo pelo novo
Logout não desloga nada	Revogou o access em vez do refresh
Refresh válido para sempre	Faltou <code>expires_at</code> ou a checagem dele
Token roubado continua funcionando	Sem rotação — refresh reutilizável indefinidamente
Confundir 401 e 403	401 = não autenticado; 403 = sem permissão
<code>client_secret</code> no front-end	Secret nunca vai para o navegador

O `client_secret` no código do front é o erro mais grave: qualquer um abre o DevTools e o copia. Secret vive **só no servidor**.

Checklist da aula

- [] `POST /login` devolve `access_token` e `refresh_token`
- [] Refresh token guardado no banco como `hash`, com `expires_at`
- [] `POST /refresh` valida existência, revogação e expiração
- [] `POST /logout` revoga o refresh token
- [] Rotação: o refresh usado é invalidado a cada renovação
- [] Entende a diferença entre os dois flows de OAuth

Referências

OAuth 2.0

- [RFC 6749 — The OAuth 2.0 Authorization Framework](#)
- [oauth.net/2](#) — visão geral dos flows
- [OAuth 2.0 Simplified — Aaron Parecki](#) — guia didático

Refresh tokens e segurança

- `golang-jwt/jwt` — geração de access tokens
- [OWASP — Cheat Sheet de gestão de sessão](#)